# Embedded Opgave 2

Beskrivelse af software: Kommunikation på både UART0 og UART1

Jacob Malthe Nielsen · ATmega328PB (16 MHz) · MPLAB XC8 3.00 · H4 Embedded

## Overblik

Programmet bygger videre på Opgave 1 og bruger nu begge UART'er i ATmega328PB. Det består af tre moduler: `main.c`, `timer.c` og `uart.c`.

Interrupts gør kun det nødvendige: de sætter et flag eller gemmer et tegn. Al udskrivning sker i hovedløkken i `main()`.

- Timer 1 laver et interrupt **hvert sekund**. Det toggler en LED og sætter et flag. Når hovedløkken ser flaget, skriver den `Sekunder efter opstart: n` på UART1.
- Tekst fra PC'en på UART0 får byttet store og små bogstaver om. De konverterede tegn skrives ud på **både UART0 og UART1**.

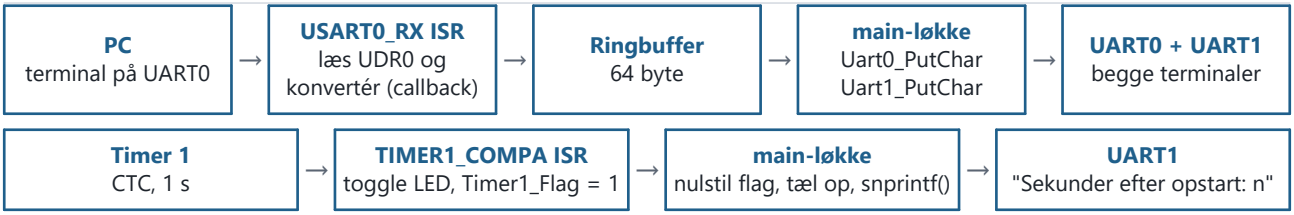
## Hardware

| Funktion | Ben | Forbundet til |
|---|---|---|
| UART0 | PD0 (RXD0) / PD1 (TXD0) | PC via mEDBG's virtuelle COM-port (USB) |
| UART1 | PB4 (RXD1) / PB3 (TXD1) | PC via CP2102: CP2102 TX → PB4, CP2102 RX ← PB3, GND ↔ GND |
| Gul LED | PB2 | Skifter tilstand hvert sekund (blinker med 0,5 Hz) |

Begge UART'er kører 9600 baud, 8N1. Der skal være en terminal åben på hver af de to COM-porte.

> **Hvorfor er LED'en flyttet fra PB3 til PB2?** På ATmega328PB er PB3 også TXD1, altså UART1's sendeben. Når UART1's sender slås til (`TXEN1`), overtager UART1 benet, og den grønne LED på PB3 kan ikke længere toggles. Timeren toggler derfor den gule LED på PB2. Benet er defineret som `TimerLedBit` i `ProjectDefines.h`.

## Dataflow




De to datastrømme i programmet. Interrupts gemmer kun data, og hovedløkken står for al afsendelse.

## Opstart (main.c)

1. LED-benet (PB2) sættes som output.
2. Timer 1 startes med et timeout på 1000 ms: `Timer1_Init_ms(1000)`.
3. UART0 initialiseres med `Uart0_Init()`, `ConvertReceivedChar` registreres som callback, og modtage-interruptet slås til.
4. UART1 initialiseres med `Uart1_Init()`.
5. De globale interrupts slås til (`sei()`), og `UART0 klar` / `UART1 klar` sendes til de to terminaler.

## Timer 1: flag hvert sekund (timer.c)

`Timer1_Init_ms(1000)` sætter Timer 1 i CTC-mode med prescaler 256. Ved 16 MHz tæller timeren 62 500 gange i sekundet:

```
OCR1A = (16 000 000 / 256) × 1000 / 1000 − 1 = 62 499
```

Det giver præcis ét compare match pr. sekund, og værdien kan være i timerens 16-bit register. `ISR(TIMER1_COMPA_vect)` gør kun to ting:

```
PORTB ^= (1 << TimerLedBit);   // toggle LED
Timer1_Flag = 1;                // fortæl main at et sekund er gået
```

`Timer1_Flag` er erklæret `volatile`, fordi den ændres i et interrupt og læses i `main()`. Uden `volatile` kan compileren finde på at læse variablen én gang og derefter aldrig igen.

## Hovedløkken (main.c)

Hovedløkken laver to ting hele tiden:

### 1. Sekundtæller → UART1

```
if (Timer1_Flag) {
    Timer1_Flag = 0;            // Husk at resette flaget!
    SecondsSinceStartup++;
    snprintf(TextBuffer, sizeof(TextBuffer),
             "Sekunder efter opstart: %lu\r\n",
             (unsigned long)SecondsSinceStartup);
    Uart1_PutString(TextBuffer);
}
```

Flaget nulstilles som det første. Hvis der kommer et nyt sekund-interrupt, mens linjen bliver sendt, går det derfor ikke tabt. Tælleren er 32 bit, så den løber først over efter ca. 136 år. Udskriften på UART1 ser sådan ud:

```
Sekunder efter opstart: 1
Sekunder efter opstart: 2
Sekunder efter opstart: 3
...
```

### 2. Tegn fra UART0 → UART0 og UART1

```
while (Uart0_GetReceivedChar(&ch)) {
    Uart0_PutChar(ch);
    Uart1_PutChar(ch);
}
```

Så længe der ligger tegn i UART0's modtagebuffer, hentes de og sendes ud på begge UART'er.

## UART-driveren (uart.c)

- **UART0:** `Uart0_Init()` sætter baudraten (`UBRR0 = F_CPU / (16 × 9600) − 1 = 103`), slår modtager og sender til og vælger 8 databit og 1 stopbit. `printf()` skriver på UART0, fordi `uart_putch` er sat som stdout.
- **UART1:** `Uart1_Init()` gør det samme med UART1's registre (`UBRR1`, `UCSR1A/B/C`, `UDR1`). `Uart1_PutString()` sender en hel tekststreng tegn for tegn.
- **Afsendelse** (`Uart0_PutChar` / `Uart1_PutChar`) venter, til senderegisteret er tomt (flaget `UDREn`), før næste tegn skrives i `UDRn`.
- **Modtagelse på UART0:** `ISR(USART0_RX_vect)` læser tegnet, kalder callback-funktionen (bogstavkonverteringen) og lægger det konverterede tegn i en **ringbuffer** på 64 byte. Interruptet skriver ved `RxHead`, og main læser ved `RxTail`. Indeksene er 8 bit, så de kan læses og skrives uden at blive afbrudt halvvejs. Derfor kan ISR og main dele bufferen sikkert uden at slå interrupts fra. Er bufferen fuld, smides tegnet væk.

### Hvorfor sender interruptet ikke selv tegnet videre?

I Opgave 1 sendte ISR'en tegnet direkte retur ved at skrive i `UDR0`. Nu skal tegnet også ud på UART1, og UART1 bruges samtidig af hovedløkken til sekundteksten. Hvis både ISR og main skrev i `UDR1`, kunne tegn blive overskrevet eller blandet ind midt i en linje.

Når al afsendelse sker fra hovedløkken, er der kun ét sted i koden, der skriver til hver UART. Samtidig bliver interruptet kort.

## Bogstavkonvertering (ConvertReceivedChar i main.c)

Konverteringen er uændret fra Opgave 1. Store og små bogstaver i ASCII adskiller sig kun ved bit 5 (`0x20`), så en XOR med `0x20` bytter dem om: `a` → `A` og `A` → `a`. Intervallerne `0x41–0x5D` og `0x61–0x7D` dækker også `[ \ ]` ↔ `{ | }`, som i dansk 7-bit ASCII er **Æ Ø Å** ↔ **æ ø å**. Andre tegn sendes uændret videre.

## Resultat

- Den gule LED på PB2 skifter tilstand hvert sekund.
- Terminalen på **UART1** (CP2102) viser `Sekunder efter opstart: n` én gang i sekundet.
- Det, der skrives i terminalen på **UART0** (mEDBG), kommer tilbage med store og små bogstaver byttet om. Det vises i både UART0-terminalen og UART1-terminalen.
- Et tegn, der bliver skrevet lige før en sekundlinje, havner i starten af den linje på UART1, fx `ASekunder efter opstart: 12`. Det sker, fordi UART1-udskriften er én fælles strøm. En linje bliver dog aldrig delt midt over.